

Class 5: Data Visualization with ggplot2

Omar Siddiqui (PID: A19093654)

2025-04-15

Overview

In this lab we explore data visualization using the **ggplot2** package in R. Every ggplot requires at minimum: a dataset (`ggplot()`), aesthetic mappings (`aes()`), and a geometric layer (e.g. `geom_point()`).

Loading ggplot2

We load ggplot2 before use. Note: `install.packages()` is run once in the console only — never left uncommented in a document.

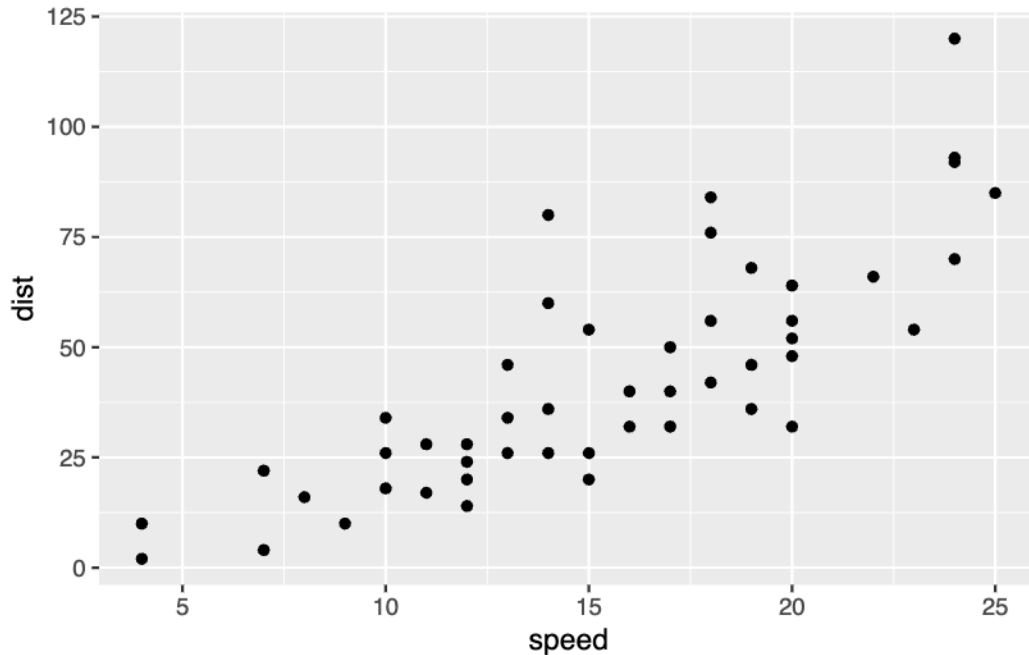
```
# install.packages("ggplot2")  
library(ggplot2)
```

Section 6: Creating Scatter Plots

First `geom_point()` plot — cars dataset

The built-in `cars` dataset has two variables: `speed` and `dist` (stopping distance).

```
# Basic scatter plot of cars dataset using ggplot + aes + geom_point  
ggplot(cars) +  
  aes(x = speed, y = dist) +  
  geom_point()
```



Each point represents a car. Faster cars take longer to stop — a clear positive relationship.

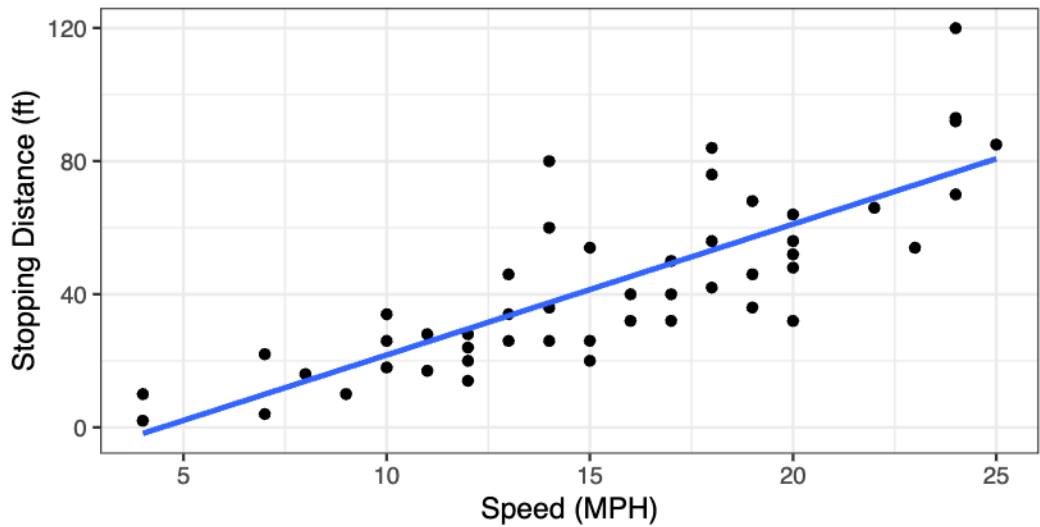
Adding multiple geoms — `geom_point()` + `geom_smooth()`

We can layer geoms. Adding `geom_smooth()` on top of `geom_point()` shows the trend line.

```
# Two geoms: points + linear model trend line
ggplot(cars) +
  aes(x = speed, y = dist) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE) +
  labs(
    title = "Speed and Stopping Distances of Cars",
    subtitle = "Linear model fit with geom_smooth(method='lm')",
    x = "Speed (MPH)",
    y = "Stopping Distance (ft)",
    caption = "Dataset: 'cars'"
  ) +
  theme_bw()
```

Speed and Stopping Distances of Cars

Linear model fit with `geom_smooth(method='lm')`



Dataset: 'cars'

`method = "lm"` fits a straight linear model line. `se = FALSE` removes the shaded confidence ribbon.

Section: Reading an Input File

We read in the DESeq2 differential expression results using `read.delim()`.

```
# Read gene expression data from course URL using read.delim()
url <- "https://bioboot.github.io/bimm143_S20/class-material/up_down_expression.txt"
genes <- read.delim(url)

# Preview the data
head(genes)
```

	Gene	Condition1	Condition2	State
1	A4GNT	-3.6808610	-3.4401355	unchanging
2	AAAS	4.5479580	4.3864126	unchanging
3	AASDH	3.7190695	3.4787276	unchanging
4	AATF	5.0784720	5.0151916	unchanging
5	AATK	0.4711421	0.5598642	unchanging

```
6 AB015752.4 -3.6808610 -3.5921390 unchanging
```

```
# How many genes are in this dataset?  
nrow(genes)
```

```
[1] 5196
```

```
# How many columns?  
ncol(genes)
```

```
[1] 4
```

```
colnames(genes)
```

```
[1] "Gene"          "Condition1" "Condition2" "State"
```

```
# How many up/down/unchanging genes?  
table(genes$State)
```

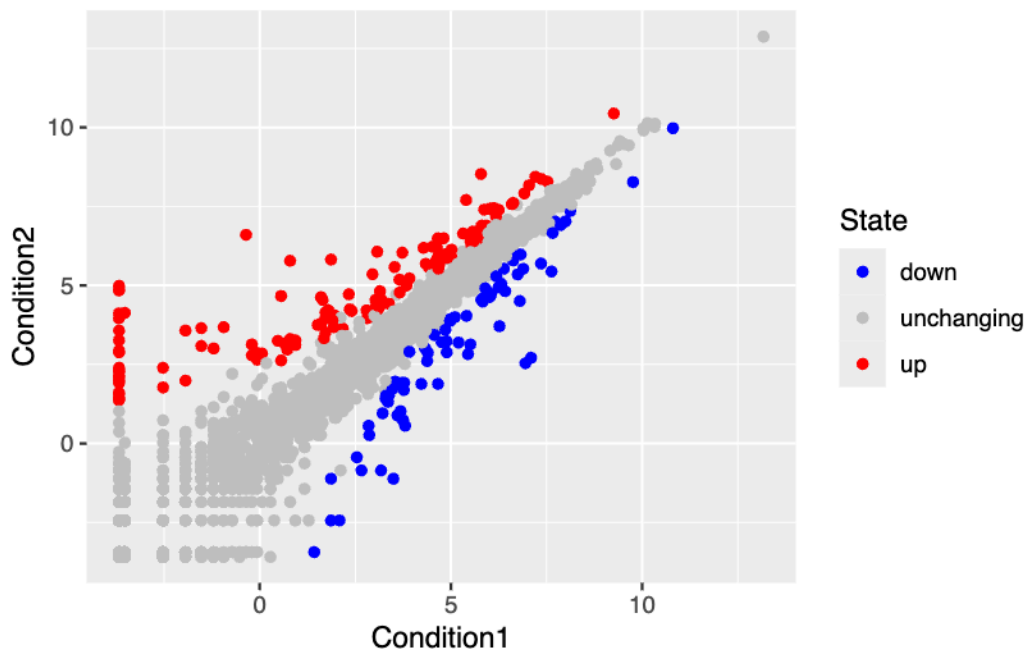
down	unchanging	up
72	4997	127

There are 5196 genes total. The `State` column classifies each gene as up-regulated, down-regulated, or unchanging.

Section: Plot with Custom Color Settings

Map the `State` column to color using `scale_colour_manual()` for explicit control over colors.

```
# Store base plot as object p  
p <- ggplot(genes) +  
  aes(x = Condition1, y = Condition2, col = State) +  
  geom_point()  
  
# Add custom colors with scale_colour_manual()  
p + scale_colour_manual(values = c("blue", "gray", "red"))
```



Blue = down-regulated, gray = unchanging, red = up-regulated.

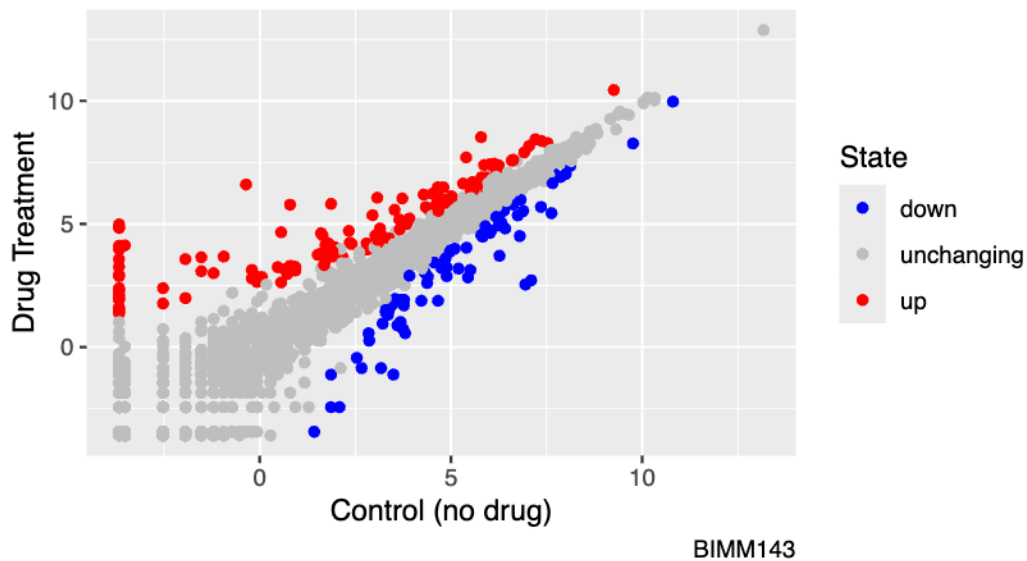
Section: Plot with labs() Settings

Add descriptive labels to the plot using `labs()`.

```
# Final polished plot with custom colors and labs() annotations
p + scale_colour_manual(values = c("blue", "gray", "red")) +
  labs(
    title      = "Gene Expression Changes Upon Drug Treatment",
    subtitle   = "Just another scatter plot made with ggplot",
    x          = "Control (no drug)",
    y          = "Drug Treatment",
    caption    = "BIMM143"
  )
```

Gene Expression Changes Upon Drug Treatment

Just another scatter plot made with ggplot



Section 7: Gapminder Dataset

The `gapminder` dataset contains economic and demographic data across countries since 1952.

```
# install.packages("gapminder")
# install.packages("dplyr")
library(gapminder)
library(dplyr)

# Filter to year 2007
gapminder_2007 <- gapminder %>% filter(year == 2007)

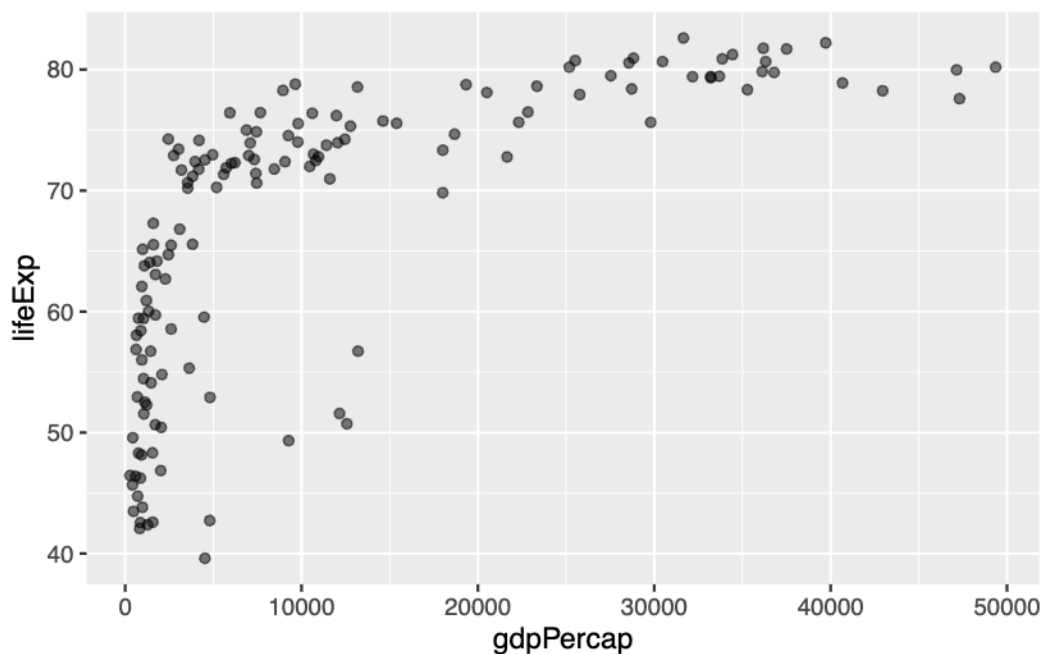
head(gapminder_2007)
```

```
# A tibble: 6 x 6
  country    continent  year lifeExp      pop gdpPercap
  <fct>      <fct>    <int> <dbl>   <int>   <dbl>
1 Afghanistan Asia      2007  43.8 31889923    975.
2 Albania    Europe    2007  76.4  3600523   5937.
```

3	Algeria	Africa	2007	72.3	33333216	6223.
4	Angola	Africa	2007	42.7	12420476	4797.
5	Argentina	Americas	2007	75.3	40301927	12779.
6	Australia	Oceania	2007	81.2	20434176	34435.

Basic gapminder scatter plot

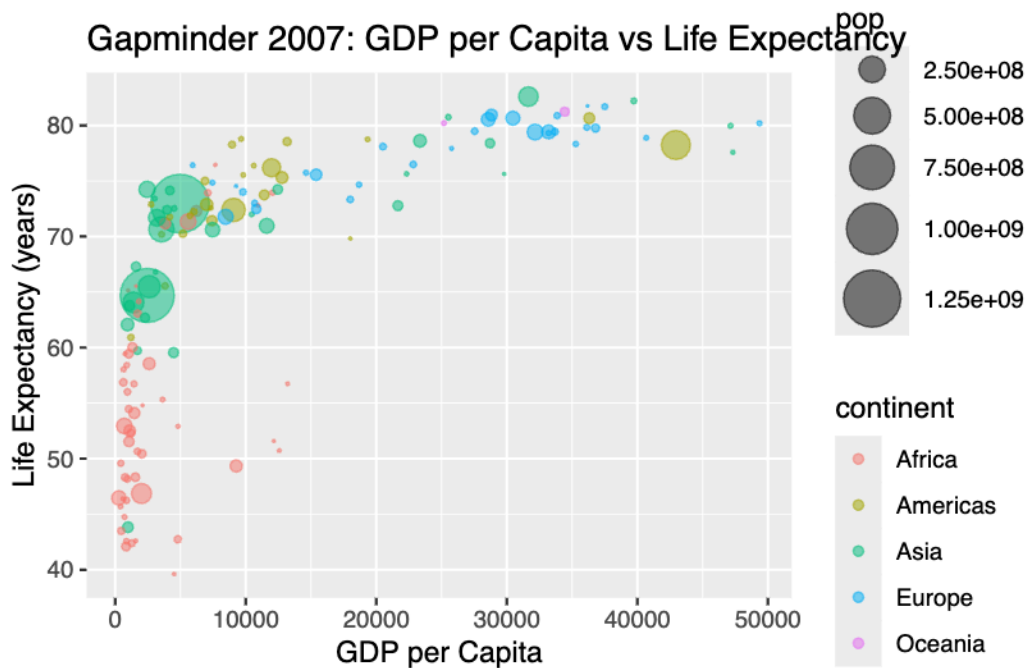
```
# GDP per capita vs life expectancy for 2007
ggplot(gapminder_2007) +
  aes(x = gdpPercap, y = lifeExp) +
  geom_point(alpha = 0.5)
```



Multi-variable plot with color and size

```
# Map continent to color, population to point size
ggplot(gapminder_2007) +
  aes(x = gdpPercap, y = lifeExp, color = continent, size = pop) +
  geom_point(alpha = 0.5) +
  scale_size_area(max_size = 10) +
```

```
labs(
  title = "Gapminder 2007: GDP per Capita vs Life Expectancy",
  x     = "GDP per Capita",
  y     = "Life Expectancy (years)"
)
```



Each point is a country. Color = continent, size = population. Wealthier countries tend to have higher life expectancy.

Section 8 (Optional): Bar Charts

Creating bar charts with `geom_col()`

```
# Get top 5 most populous countries in 2007
gapminder_top5 <- gapminder %>%
  filter(year == 2007) %>%
  arrange(desc(pop)) %>%
  top_n(5, pop)
```



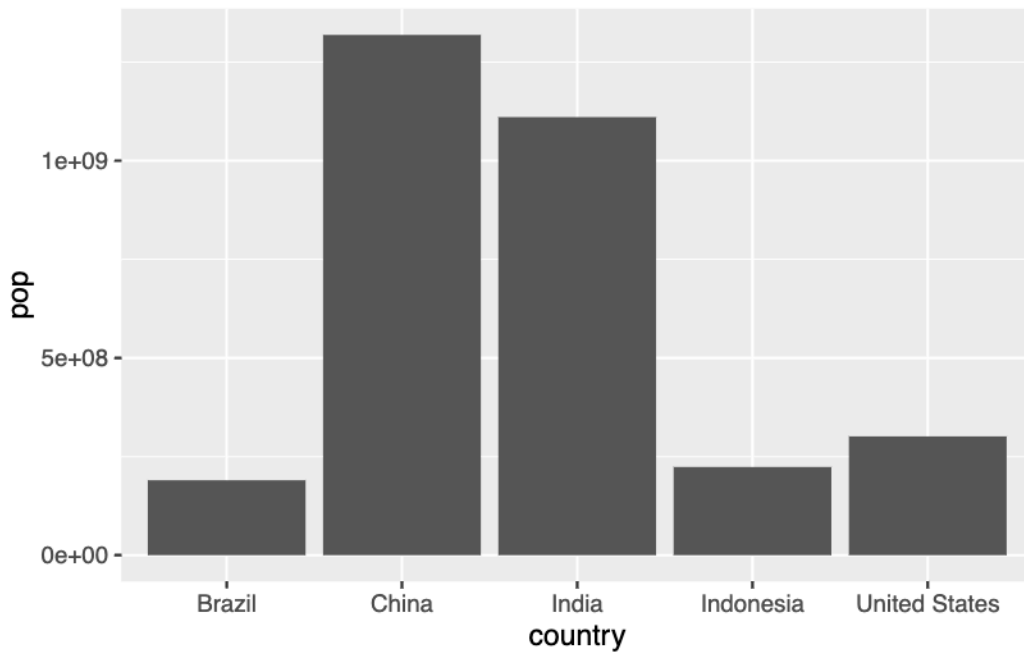
```
gapminder_top5
```

```
# A tibble: 5 x 6
```

	country	continent	year	lifeExp	pop	gdpPercap
	<fct>	<fct>	<int>	<dbl>	<int>	<dbl>
1	China	Asia	2007	73.0	1318683096	4959.
2	India	Asia	2007	64.7	1110396331	2452.
3	United States	Americas	2007	78.2	301139947	42952.
4	Indonesia	Asia	2007	70.6	223547000	3541.
5	Brazil	Americas	2007	72.4	190010647	9066.

```
# Simple bar chart
```

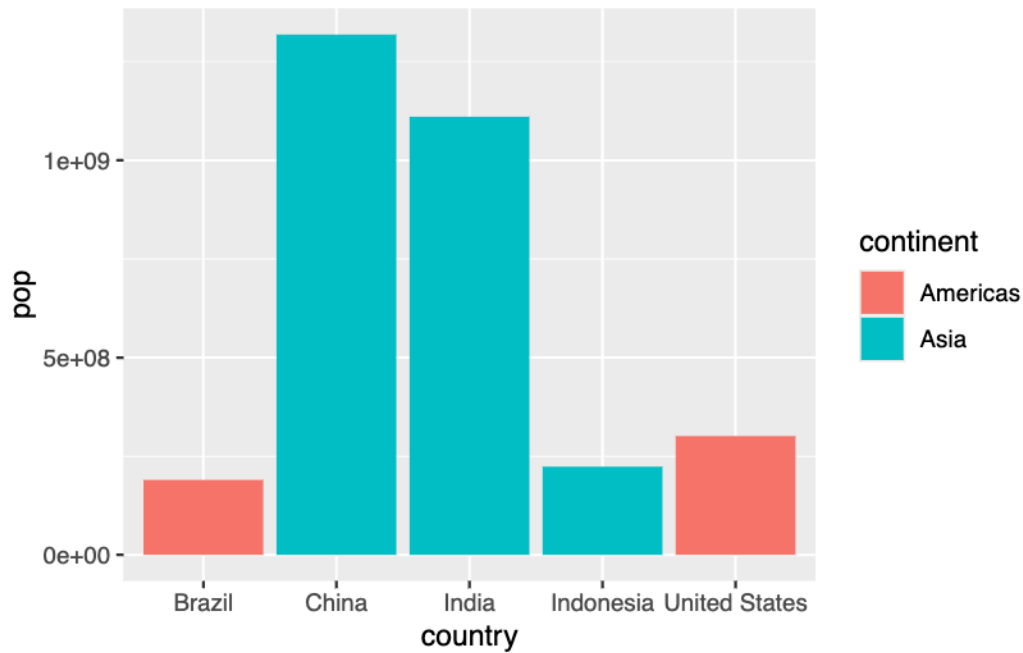
```
ggplot(gapminder_top5) +  
  geom_col(aes(x = country, y = pop))
```



Filling bars with color

```
# Fill bars by continent (categorical variable)
```

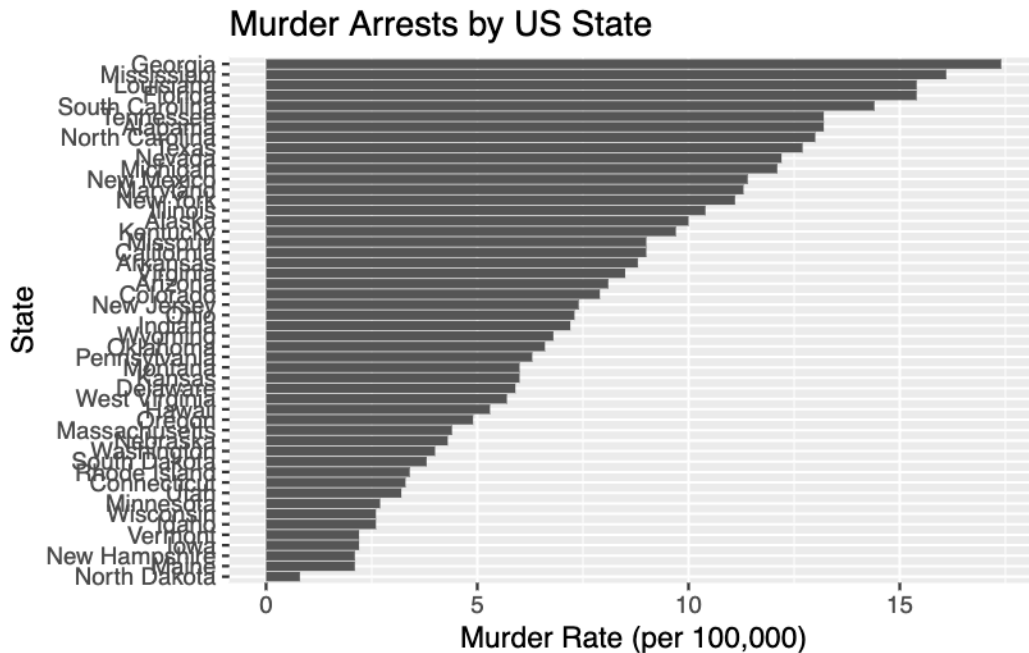
```
ggplot(gapminder_top5) +  
  geom_col(aes(x = country, y = pop, fill = continent))
```



Flipping bar charts with coord_flip()

```
# US Arrests dataset - reorder states by murder rate and flip
USArrests$State <- rownames(USArrests)

ggplot(USArrests) +
  aes(x = reorder(State, Murder), y = Murder) +
  geom_col() +
  coord_flip() +
  labs(
    title = "Murder Arrests by US State",
    x      = "State",
    y      = "Murder Rate (per 100,000)"
  )
```



`coord_flip()` rotates the chart horizontal, making state labels readable.

Session Info

```
sessionInfo()
```

```
R version 4.5.3 (2026-03-11)
Platform: aarch64-apple-darwin20
Running under: macOS Tahoe 26.4.1
```

```
Matrix products: default
```

```
BLAS: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRblas.0.dylib
LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;
```

```
locale:
```

```
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
```

```
time zone: America/Los_Angeles
```

```
tzcode source: internal
```

attached base packages:

```
[1] stats      graphics  grDevices  utils      datasets  methods   base
```

other attached packages:

```
[1] dplyr_1.2.1      gapminder_1.0.1 ggplot2_4.0.2
```

loaded via a namespace (and not attached):

```
[1] vctrs_0.7.2      nlme_3.1-168      cli_3.6.5         knitr_1.51
[5] rlang_1.1.7      xfun_0.57         generics_0.1.4    S7_0.2.1
[9] jsonlite_2.0.0   labeling_0.4.3    glue_1.8.0        htmltools_0.5.9
[13] scales_1.4.0     rmarkdown_2.31    grid_4.5.3        evaluate_1.0.5
[17] tibble_3.3.1     fastmap_1.2.0     yaml_2.3.12       lifecycle_1.0.5
[21] compiler_4.5.3   RColorBrewer_1.1-3 pkgconfig_2.0.3    mgcv_1.9-4
[25] lattice_0.22-9   farver_2.1.2      digest_0.6.39     R6_2.6.1
[29] utf8_1.2.6       tidyselect_1.2.1  splines_4.5.3     pillar_1.11.1
[33] magrittr_2.0.5   Matrix_1.7-4      withr_3.0.2       tools_4.5.3
[37] gtable_0.3.6
```